

Reviewer Pack — Minimal Local Induction Dimension in Boolean Algebra and Com...

Entry fa741588a50c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 11:58:39 UTC

Minimal Local Induction Dimension in Boolean Algebra and Communication Complexity Rank

Entry ID: fa741588a50c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 11:58:39 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Local Induction Dimension in Boolean Algebra) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

The minimal local induction dimension of the clause indicator polynomial of a CNF is linearly correlated with its communication complexity rank, such that $\text{mld}(\text{CNF}) = \Theta(\text{rk_comm}(\text{CNF}))$ for all CNFs with at most 40 clauses.

Rationale (proposer's reasoning):

Local induction dimension (LID) in Boolean algebra measures the complexity of generating all possible outputs from given inputs. Communication complexity rank is a measure of the minimum number of bits required to communicate an output between two parties. Both are structural invariants and might reveal underlying complexities in computational tasks. The conjecture suggests that these two invariants could be related, potentially providing insights into proving lower bounds for communication complexity.

Taxonomy category: LID_COMM_RANK_CORRELATION (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c5d63d8c95c99b6b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean correlation coefficient between minimal local induction dimension (mld) and communication complexity rank (rk_comm) for all CNFs with at most 40 clauses is greater than or equal to 0.8, and the standard deviation of mld values across seeds does not exceed 2.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Minimal Local Induction Dimension" AND "Boolean Algebra" AND communication complexity - "Clause Indicator Polynomial" AND "Communication Complexity Rank" AND CNF - "mld(CNF) = $\theta(\text{rk_comm}(\text{CNF}))$ " AND "local induction dimension" AND "matrix rank"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(random.randint(1, n))]
            clauses.append(clause)
        return clauses

    def clause_indicator_polynomial(cnf):
        n = len(cnf[0])
        p = [[0] * (2**n) for _ in range(n)]
        for i in range(n):
            for j in range(2**i):
                for k in range(1 << (n - i - 1)):
                    if all((j & (1 << m)) == ((k + j) & (1 << m)) for m in range(i)):
                        p[i][j] += 1
        return p

    def minimal_local_induction_dimension(p):
        n = len(p)
        dim = 0
        for i in range(n):
            if any(p[i][j] != 0 for j in range(2**i)):
                dim += 1
        return dim
```

```

def communication_complexity_rank(cnf):
    n = len(cnf[0])
    m = len(cnf)
    matrix = [[0] * (n + 1) for _ in range(m)]
    for i, clause in enumerate(cnf):
        for var in clause:
            if var > 0:
                matrix[i][var - 1] = 1
            else:
                matrix[i][-1] += 1
    rank = 0
    for row in matrix:
        if any(x != 0 for x in row):
            rank += 1
    return rank

n_max = 40
instances_tested = 0
mld_values = []

for n in range(5, n_max + 1):
    cnf = generate_cnf(n)
    p = clause_indicator_polynomial(cnf)
    mld = minimal_local_induction_dimension(p)
    rk_comm = communication_complexity_rank(cnf)

    if mld is not None and rk_comm is not None:
        instances_tested += 1
        mld_values.append(mld)

if instances_tested == 0:
    return {
        "metric_name": "mld vs. rk_comm",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mean_mld = sum(mld_values) / instances_tested
std_mld = math.sqrt(sum((x - mean_mld) ** 2 for x in mld_values) / instances_tested)
correlation_coefficient = sum((mld_values[i] - mean_mld) * (i + 5 - 15) for i in range(instances_tested)) / instances_tested

return {
    "metric_name": "mld vs. rk_comm",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": correlation_coefficient >= 0.8 and std_mld <= 2,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys

```

```

seeds = [int(x) for x in sys.argv[1:]] or [random.randint(1, 1000) for _ in range(30)]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_587ae82b.py", line 105, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_587ae82b.py", line 70, in run_trial
    rk_comm = communication_complexity_rank(cnf)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_587ae82b.py", line 53, in communication_complexity_rank
    matrix[i][var - 1] = 1
    ~~~~~~^~~~~~
IndexError: list assignment index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from evaluating the mean correlation coefficient and standard deviation of minimal local induction dimension (mld) values as required by the support condition. | next: Investigate the cause of the crash in

the test code and attempt to run the test again to gather the necessary data for evaluating the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14068
2	preregistration	ollama_remote	glm4:latest	0	0	9397
3	novelty	ollama_remote	glm4:latest	0	0	8525
4	novelty	ollama_remote	glm4:latest	0	0	10174
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20323
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	34923
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18473
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13695
9	judge	ollama_remote	glm4:latest	0	0	50682

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 180260 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/fa741588a50c.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/fa741588a50c.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/fa741588a50c.tar.gz* (if generated)